# CS-A1150 TIETOKANNAT

# Project 1

# 1. UML

UML diagram is added at the end of the file

## 2. Relation schemas converted from the UML diagram.

Repair(repairName, OrderID, LicenNum, EmployeeID, Duration total)

RegularMaintenance( Distance, OrderID, LicenNum, EmployeeID, DurationTotal)

Operation(Duration, OperationName, Distance, ReqDevice)

SparePart(OrderID , PartNumber, Amount, Sum)

Invoice(OrderID, CustomerID, EmployeID, Hours, Deadline, Status, Sum, RemindersNo)

Reminder(IOrderID, Extra, NewSum)

Car(LicenseNumber, CustomerID, Distance)

Owner(LicenseNumber, Name)

Device(DeviceID, DeviceType, AvalibilityDateAndTime)

Employe(ID, Name, AvalibilityDataAndTime)

Absences(EmployeID, From, To)

Reservation of operation(OrderID, CustormerID, DeviceID, EmployeID, From, To)

Customer(CustomerID, , Name, Phone, Email, Address)

Brought(CustomerID, LicenseNum)

works on(employeID, OrderID)

Has operations(OperationName, OrderID)

## 3. Explanation of the solution

**• The garage performs both regular maintenances and repairs of sudden faults.** We have two classes: repair and regularMaintenance.

**• A regular maintenance includes a list of operations ordered by the manufacturer (e.g. checking the timing belt or changing the gas filter) which depend on the number of kilometers driven with this car. (There are different lists for the regular maintenance of 30 000 km and 90 000 km, for example.) To simplify the project, we assume that all operations belonging to a regular maintenance and their duration are known beforehand.** RegularMaintenance gets Distance from Car and gives it to Operation, where we get operation name and duration based on distance.

**• The repair of a sudden fault also consists of operations. To simplify the project, we assume (although it is not realistic) that all operations belonging to a certain repair and their duration are known beforehand.** In class Operation we duration attribute which is used to aquire DurationTotal for maintinances. The connection is maintained via OrderID key attribute.

**• A maintenance is performed non-stop by the same employee from the beginning to the end.** Maintenance related only to one Employee, the assosiation maintained via works on class.

**• Some of the operations may require devices which the garage has a limited number (a crane, for example). However, the garage may own several items of the same device (for example, three identical cranes).** We expected that all items in garage have own ID, but they can have same DeviceType as crane.

**• The garage maintains and repairs only cars. The cars are identified by the license number. The garage knows the owner(s) of the car. The car may have several owners (for example, a couple owning a car to together). The customer who has brought the car to the maintenance may be different from the owner.** We don't have motorcycle-class. Classes Car, Customer and Owner are assosiated via LicenseNumber. Class owner can have several names with same LicenseNumber. Customer can be found via License Number as well.

**• The customer may reserve a time for a maintenance beforehand. The time is given such that the garage has the devices needed and an employer for the maintenance at the given time. The employer is reserved for the duration of the maintenance, but the devices only for the duration of the operations where they are really needed.** Customer class is associated with the Reservation of operation

class. A maintenance reservation automatically decompositioned into operations according to OrderID attribute. Then an employee and devices are reserved for each operation. Reservation of the employee separately for each operation should improve time management and efficiency of the worker.

• **Spare parts may be needed to carry out the maintenance.** We have class SparePart.

• **An invoice of the maintenance depends on the number of working hours needed and which spare parts where needed.** Invoice, SparePart, Repair and RegularMaintenance have attribute OrderID. Via it we can find all parts were used for this job and calculate sum and find attribute DurationTotal.

• **If the customer does not pay the bill on time, a payment reminder is sent to the customer. If necessary, several reminders may be sent. Each reminder may add an extra cost to the original sum.** We have class Reminder which is connected to Invoice via OrderID. Reminder has attribute NewSum.

• **Store the list of operations which belong to a certain regular maintenance or repair and information about which devices are needed in which operations. About the devices only the type of the device is stored.** Maintenance and Operation classes are assosiated via has operations class. Operation class has ReqDevice attribute which indicated the type of required device and maintenance an association with the Device class.

• **Store information about which employee is reserved for a certain maintenance at a certain time and which devices are reserved for the operations of the maintance. In the reservations, it is specified which of the identical items is reserved. For a certain maintenance, a planned starting and end times are stored** Attributes From and To in Reservation of operation class are indicating date and time when reservation for an operation takes place. For each operation, a worker and a device are reserved and asociated with EmpolyeID and DeviceID and stored. The full time for reservation of worker for a certain maintenance can be found with help of OrderID attribute in Reservation of operation class.

• **Search for a time when some of the empolyees is free long enough (non-stop) for a wanted maintenance and reserve an employee for this maintenance,** Reservation of operation class stores all reservations of workers with employeID attribute. The times that are not in between of From and To attributes for a given deviceID are free times of the employe.

• **Search for a time when a device needed in a operation belonging to a maintenance is free and reserve the device for this maintenance. (The device is reserved only for the duration of the operation, not for the duration of the whole maintenance.) This concerns only the devices whose number is limited. We assume that there are enough basic tools such that no reservations are needed for them** Reservation of operation class stores allows reservations of devices with deviceID attribute. The times that are not in between of From and To attributes for a given deviceID are times when device is not reserved.

• **Save information about vacations and other absences of the employees known beforehand. The**
**employee cannot be reserved for a maintenance while he/she is known to be absent.** Class Absences is responsible for that. It is connected to Employee via EmployeID. Employe.AvailabilityDateAndTime gets updated by Absences.To.

• **Store information about cars brought to garage, their owners and customers who brought the car**
**to the garage.** Cars, Owner and Customer classes are connected via LicenseNo.

• **Find out which maintenances has been performed to a certain car in this garage before.** By connecting Repair and Regular Maintenance and selecting rows where LicenNum="Our cars number" we can find it's history.

• **Find out contact information about a customer who brought the car to the garage.** Customer and Car classes are assosited via Brought class. Customer class have attributes for such contacts as Phone, Mail and Address

• **Store information about the things that affect the amount of the invoice: needed spare parts and**
**the number of them and the number of working hours an employee (and who) has used for the**
**maintenance.** Via OrderID we can find all SpareParts needed for this job. SpareParts has attributes Amount and Sum. Same way we find maintenance with attribute DurationTotal and via EmployeID we find Emloye and his name.

• **Store and search for the information about sending and paying of the bills and the possible**
**reminders. The system must not restrict the number of reminders which can be sent about one**
**bill.** Invoice has several attributes for searching. Reminders are connected to invoice vie OrderID and their amount isn't restricted.

• **Search for customers having bills which have not been paid.** Can be found by connecting  Invoice and Customer via CustomerID and selecting rows where status=”unpaid”.

• **Search for customers who have been sent a certain number of reminders but who has not yet**
**paid the bill.** Can be found by connecting Invoice and Customer via CustomerID and selecting rows where status=”unpaid” AND RemindersNo =”needed number”.

4. Answers To Questions

## What are the functional dependencies of the database? :

LicenseNumber-> CustomerID, Customer.Name, Customer. Phone, Customer. Mail, Customer. Address, Distance

OrderID -> Deadline, Hours, CustomerID, Status, EmployeID, Sum, RemindersNo, LicenseNumber

EmployeID-> Employe.Name, Employe.AvailabilityDateAndTime,

DeviceID-> DeviceType, Device. AvailabilityDateAndTime.

## Anomaly analysis

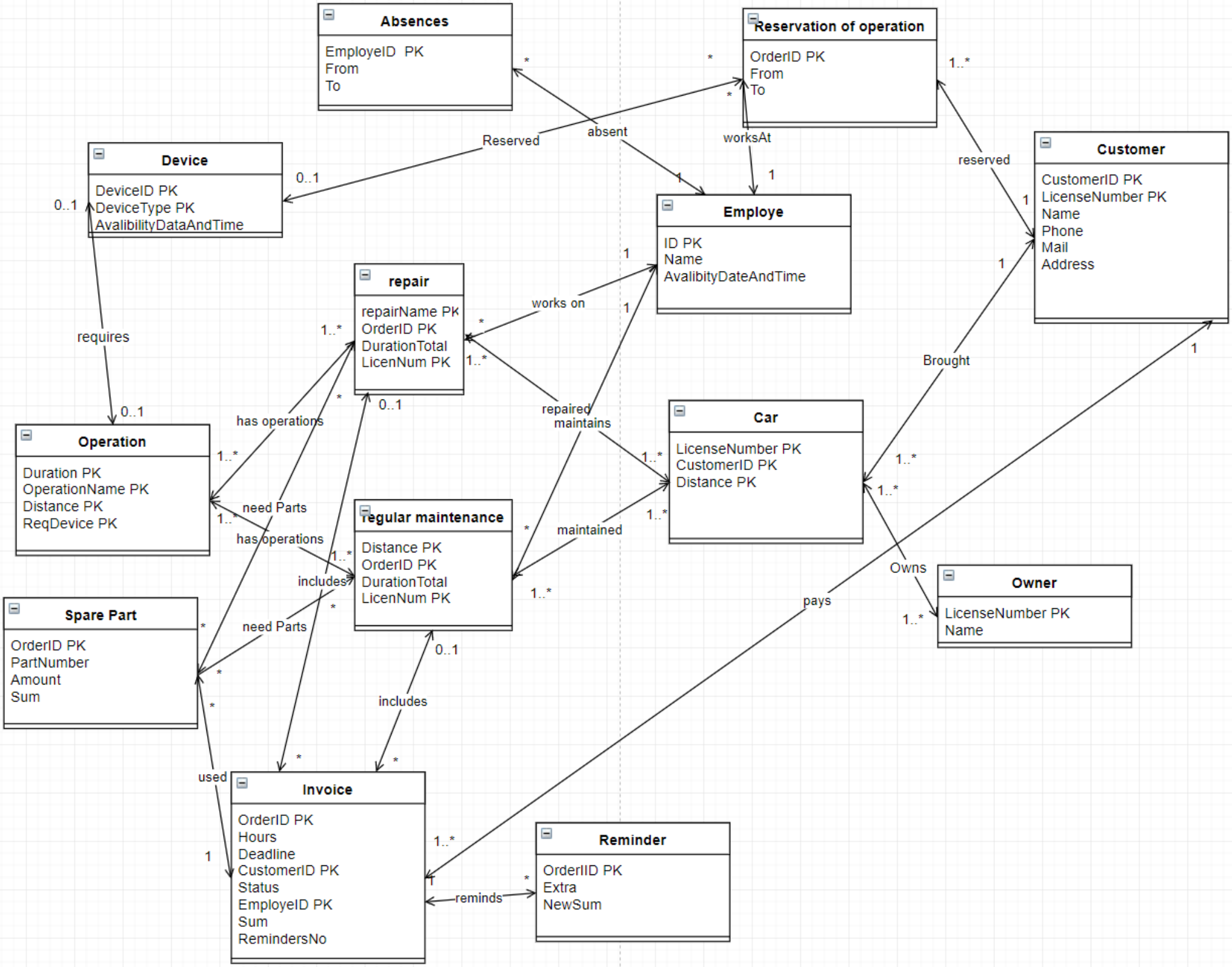
• Redundancy: The database has no repeating attributes in classes aside from key attributes which are responsible for functional dependencies. Attribute AvalibityDateAndTime from Device and Employe classes along with DurationTotal from repair and regular maintenance can be seen as some form of redundancy. The attributes are expected to be updated with an attribute To from Reservation of operation and Absences classes. However, the AvalibityDateAndTime with DurationTotal were made to improve Quality of Life and are not critical to the functioning of the database and if the procedures are properly followed will work fine.

• Update anomalies: Again, except key attributes such as OrderID classes does not have repeating attributes. The same issue as before arise with AvalibityDateAndTime and DurationTotal, the procedure for updating the attributes should be properly followed.

• Deletion anomalies: The database should not have deletion anomalies, as attributes are propely separated into classes that will represent them well without mixing several things into the same class.

## Boyce-Codd

The database follows BCNF, as when using attributes of the functional dependencies the database does not repeating data. Also for all nontrivial functional dependencies there are distinct superkeys.


Absences
EmployeID PK
From
To
Reservation of operation
OrderID PK
From
To
Reserved
absent
worksAt
reserved
Device
DeviceID PK
DeviceType PK
AvalibilityDataAndTime
Employe
ID PK
Name
AvalibityDateAndTime
Customer
CustomerID PK
LicenseNumber PK
Name
Phone
Mail
Address
repair
repairName PK
OrderID PK
DurationTotal
LicenNum PK
works on
requires
Brought
repaired
maintains
Operation
Duration PK
OperationName PK
Distance PK
ReqDevice PK
has operations
need Parts
has operations
Car
LicenseNumber PK
CustomerID PK
Distance PK
maintained
regular maintenance
Distance PK
OrderID PK
DurationTotal
LicenNum PK
includes
Owns
Owner
LicenseNumber PK
Name
pays
Spare Part
OrderID PK
PartNumber
Amount
Sum
need Parts
includes
used
Invoice
OrderID PK
Hours
Deadline
CustomerID PK
Status
EmployeID PK
Sum
RemindersNo
reminds
Reminder
OrderIID PK
Extra
NewSum